PAWFINDER: COMMUNITY PET ADOPTION & FOSTER MANAGEMENT SYSTEM

A DBMS Mini Project Report

Course: Database Management Systems (DBMS)
Database Engine: PostgreSQL 15+ (Hosted on Supabase)
Application Layer: Next.js 16 (React 19, TypeScript, Tailwind CSS v4)
Target Domain: Community Pet Welfare & Foster Placement

Table of Contents

1. Introduction
   1.1 Objectives
   1.2 Scope
2. System Requirements
   2.1 Functional Requirements
   2.2 Non-Functional Requirements
   2.3 Tools & Technology Stack
3. Entity-Relationship (ER) Diagram
   3.1 Relationship Summary
4. Entity & Attribute Description
   4.1 Users
   4.2 Species
   4.3 Pets
   4.4 Adoption Requests
5. Normalization
   5.1 First Normal Form (1NF)
   5.2 Second Normal Form (2NF)
   5.3 Third Normal Form (3NF) & BCNF
   5.4 Design Trade-offs & Integrity Guarantees
6. Relational Schema (DDL)
7. Sample Queries & Views
   7.1 Available Inventory View
   7.2 Filter by Species with Total Counts
   7.3 In-Flight Adoption Requests by Owner
   7.4 Completed Adoption Handover Analytics
8. Advanced DBMS Concepts: Views & Integrity Constraints
9. Application Implementation & Workflows
10. Conclusion & Future Enhancements

1. Introduction

PawFinder is a relational database management system designed to coordinate community pet adoptions and foster placements. Animal welfare rescue groups, community shelters, and pet owners often rely on disparate chat groups or unorganized social posts, which causes duplicated applications, untracked animal availability, and lack of follow-up accountability.

1.1 Objectives

- Design a normalized relational schema complying with Boyce-Codd Normal Form (BCNF).
- Model one-to-many relationships with full referential integrity and cascading deletions.
- Enforce domain rules at the database engine level using SQL CHECK and UNIQUE constraints.
- Provide SQL views to abstract complex multi-table joins for frontend client applications.
- Connect the relational database to a modern full-stack web application with type-safe queries.

1.2 Scope

The system manages user profiles (owners and prospective adopters), taxonomic species categories, animal listings with age and health status, and adoption transaction requests with preferred dates and suitability notes.

2. System Requirements

2.1 Functional Requirements

- Users can browse available pets filtered dynamically by species category.
- Pet owners can list animals for adoption with name, species, breed, age in months, and descriptions.
- Prospective adopters can submit adoption requests specifying desired pickup dates and home details.
- Pet owners can review incoming requests and approve or reject applications.
- Upon approval, the pet's status transitions to 'Adopted', and the owner can mark the adoption as completed.

2.2 Non-Functional Requirements

- Data Integrity: Foreign key constraints prevent orphaned pet or request records.
- Domain Safety: Check constraints restrict pet status to valid states and disallow negative ages.
- Performance: Queries utilize primary keys and relational views for sub-second retrieval.
- Reliability: Cloud PostgreSQL hosting guarantees persistent storage and transactional durability.

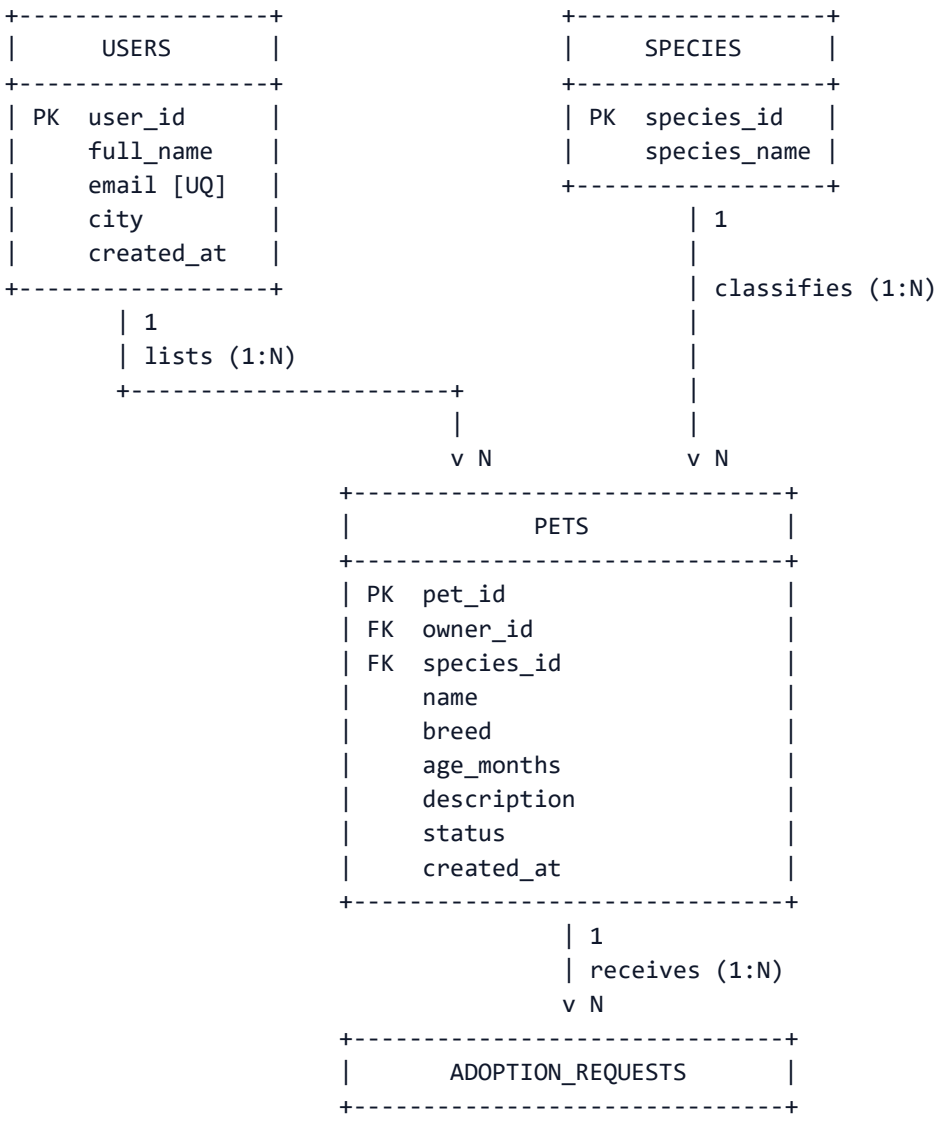
2.3 Tools & Technology Stack

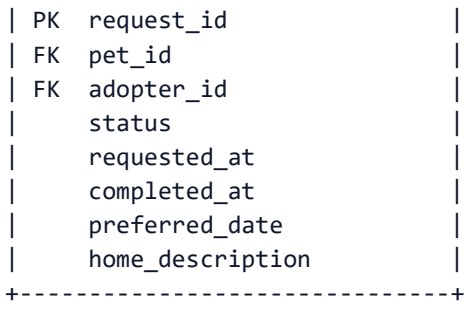
Component	Technology	Role & Description

Database Engine	PostgreSQL 15+ (Supabase)	Hosted relational database with SQL engine and auth
Frontend & API	Next.js 16 (React 19, TypeScript)	Server & Client components with dynamic streaming
Client Library	@supabase/supabase-js	Type-safe relational query builder and SDK
Styling	Tailwind CSS v4	Responsive UI with modern aesthetic and dark mode

3. Entity-Relationship (ER) Diagram

The conceptual model consists of four strong entities connected via binary one-to-many relationships:





3.1 Relationship Summary

Relationship	Entity 1	Entity 2	Cardinality & Integrity Policy
Lists / Owns	USERS	PETS	1 : N (ON DELETE CASCADE)
Categorizes	SPECIES	PETS	1 : N (ON DELETE CASCADE)
Submits	USERS	ADOPTION_REQUESTS	1 : N (ON DELETE CASCADE)
Receives	PETS	ADOPTION_REQUESTS	1 : N (ON DELETE CASCADE)

4. Entity & Attribute Description

4.1 USERS Table

Attribute	Data Type	Key	Description
user_id	SERIAL	PK	Unique system identifier for each registered user
full_name	VARCHAR(100)	No	Full legal or display name of the user
email	VARCHAR(100)	UQ	Unique contact email address
city	VARCHAR(50)	No	City of residence for local adoption matching
created_at	TIMESTAMPTZ	No	Record creation timestamp (DEFAULT

			NOW)

4.2 SPECIES Table

Attribute	Data Type	Key	Description
species_id	SERIAL	PK	Unique numeric ID for the pet taxonomy
species_name	VARCHAR(50)	UQ	Standardized name: Dog, Cat, Bird, Rabbit

4.3 PETS Table

Attribute	Data Type	Key	Description
pet_id	SERIAL	PK	Unique numeric identifier for the pet listing
owner_id	INT	FK	References users(user_id) ON DELETE CASCADE
species_id	INT	FK	References species(species_id) ON DELETE CASCADE
name	VARCHAR(150)	No	Name of the animal
breed	VARCHAR(80)	No	Specific breed or mix description
age_months	INT	No	Age in months; CHECK (age_months >= 0)
description	TEXT	No	Temperament, behavioral notes, and health info
status	VARCHAR(20)	No	CHECK (status IN ('Available', 'Adopted', 'Pending'))
created_at	TIMESTAMPTZ	No	Listing publication timestamp

4.4 ADOPTION_REQUESTS Table

Attribute	Data Type	Key	Description
request_id	SERIAL	PK	Unique transaction identifier
pet_id	INT	FK	References pets(pet_id) ON DELETE CASCADE
adopter_id	INT	FK	References users(user_id) ON DELETE CASCADE
status	VARCHAR(20)	No	CHECK (status IN ('Pending', 'Approved', 'Rejected', 'Completed'))
requested_at	TIMESTAMPTZ	No	Timestamp when request was filed
completed_at	TIMESTAMPTZ	No	Timestamp when animal was handed over
preferred_date	DATE	No	Proposed adoption / pickup date
home_description	TEXT	No	Applicant's home environment, yard, and experience

5. Normalization

5.1 First Normal Form (1NF)

All columns contain atomic, indivisible values. Repeating groups such as multiple species or multi-valued attributes have been eliminated. Each table has an explicit primary key.

5.2 Second Normal Form (2NF)

All primary keys are single-column synthetic surrogates (SERIAL PRIMARY KEY). Therefore, partial functional dependencies cannot exist, satisfying 2NF.

5.3 Third Normal Form (3NF) & BCNF

In an unnormalized design, species names might be directly embedded within the PETS table: pet_id -> species_name. This would introduce a transitive dependency via species_id -> species_name.

Decomposition: Factoring out SPECIES into (species_id, species_name) eliminates update anomalies (e.g. renaming a species in one place) and deletion anomalies (purging all cats does not delete the species 'Cat').

For every non-trivial functional dependency X -> Y across all four relations, X is a superkey. Thus, the database satisfies Boyce-Codd Normal Form (BCNF).

6. Relational Schema (DDL)

```
-- 1. USERS
CREATE TABLE IF NOT EXISTS users (
    user_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    city VARCHAR(50) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. SPECIES
CREATE TABLE IF NOT EXISTS species (
    species_id SERIAL PRIMARY KEY,
    species_name VARCHAR(50) NOT NULL UNIQUE
);

-- 3. PETS
CREATE TABLE IF NOT EXISTS pets (
    pet_id SERIAL PRIMARY KEY,
    owner_id INT NOT NULL,
    name VARCHAR(150) NOT NULL,
    species_id INT NOT NULL,
    breed VARCHAR(80),
    age_months INT CHECK (age_months >= 0),
    description TEXT,
    status VARCHAR(20) DEFAULT 'Available'
        CHECK (status IN ('Available', 'Adopted', 'Pending')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_owner FOREIGN KEY (owner_id) REFERENCES users(user_id) ON DELETE
CASCADE,
    CONSTRAINT fk_species FOREIGN KEY (species_id) REFERENCES species(species_id) ON
DELETE CASCADE
);

-- 4. ADOPTION REQUESTS
CREATE TABLE IF NOT EXISTS adoption_requests (
    request_id SERIAL PRIMARY KEY,
    pet_id INT NOT NULL,
    adopter_id INT NOT NULL,
    status VARCHAR(20) DEFAULT 'Pending'
        CHECK (status IN ('Pending', 'Approved', 'Rejected', 'Completed')),
```

```
    requested_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP WITH TIME ZONE NULL,
    preferred_date DATE NULL,
    home_description TEXT NULL,
    CONSTRAINT fk_pet FOREIGN KEY (pet_id) REFERENCES pets(pet_id) ON DELETE CASCADE,
    CONSTRAINT fk_adopter FOREIGN KEY (adopter_id) REFERENCES users(user_id) ON DELETE
CASCADE
);
```

7. Sample Queries & Views

7.1 Active Inventory View

```
CREATE OR REPLACE VIEW available_pets AS
SELECT
    p.pet_id,
    p.name,
    s.species_name,
    p.breed,
    p.age_months,
    p.status,
    u.full_name AS owner_name,
    u.email AS owner_email
FROM pets p
JOIN species s ON p.species_id = s.species_id
JOIN users u ON p.owner_id = u.user_id;
```

7.2 Adoption Statistics by Species

```
SELECT
    s.species_name,
    COUNT(p.pet_id) AS total_listed,
    COUNT(CASE WHEN p.status = 'Adopted' THEN 1 END) AS total_adopted,
    ROUND(COUNT(CASE WHEN p.status = 'Adopted' THEN 1 END)::numeric /
NULLIF(COUNT(p.pet_id), 0) * 100, 2) AS adoption_rate
FROM species s
LEFT JOIN pets p ON s.species_id = p.species_id
GROUP BY s.species_name
ORDER BY total_adopted DESC;
```

8. Advanced DBMS Concepts: Views & Integrity Constraints

PawFinder shifts business validation from transient frontend checks directly into the database engine:

- Domain Constraints: CHECK (status IN (...)) strictly limits valid lifecycle states.

- Referential Integrity: ON DELETE CASCADE guarantees no orphaned pets or requests if a parent row is deleted.
- Logical Views: Decouple physical schema changes from application select queries.

9. Application Implementation & Workflows

The frontend application was developed using Next.js 16 with React 19 and Tailwind CSS v4. Key interactive workflows include:

- User Persona Switcher: Allows evaluating user roles (owner vs. adopter) without authentication friction.
- Species Categorization: Instant multi-attribute filtering with real-time inventory count.
- Request Lifecycle: Submission with preferred date and home notes, owner review, and completion.

10. Conclusion & Future Enhancements

PawFinder successfully exemplifies modern database engineering principles. Future planned enhancements include adding image BLOB storage support, audit logging triggers for request status changes, and geolocation spatial queries for proximity matching.